

Elaborato di Ingegneria del Software

SISTEMA SOFTWARE MATEMATIKA

Studenti: Izaro

21 giugno 2026

Indice

1	Introduzione e Descrizione del Progetto	3
2	Analisi dei Requisiti	3
2.1	Requisiti Funzionali (RF) e Non Funzionali (RNF)	3
2.2	Casi d'Uso e Modello di Dominio	3
3	Piano di Test Funzionale	5
3.1	Identificazione di Parametri e Categorie	5
3.2	Casi di Test Generati	6
4	Progettazione Architetturale	6
4.1	Architettura a Componenti (Pattern BCED)	6
4.2	Diagrammi di Sequenza	7
5	Implementazione e Transizione Tecnologica	8
5.1	Deployment	10
6	Testing Strutturale e Unitario	10
6.1	Analisi Algoritmica (Control Flow Graph)	10
6.2	Vitest in Sostituzione a JUnit	11

1 Analisi e specifica dei requisiti

1.1 Revisione dei requisiti

Il progetto Matematika è un'applicazione web interattiva creata per permettere agli studenti di esplorare la matematica (geometria) come un grafo di dipendenze logiche, superando i limiti dei testi lineari tradizionali. Il sistema elabora l'albero delle dipendenze tra assiomi e teoremi per valutarne la correttezza in tempo reale.

1.2 Classificazione dei requisiti

Di seguito la tabella riassuntiva dei requisiti funzionali e non funzionali estratti dall'analisi:

ID	Descrizione
RF01	Il sistema deve permettere la visualizzazione testuale e grafica di assiomi, definizioni e teoremi.
RF02	L'utente deve poter attivare o disattivare gli assiomi (Modalità Sandbox).
RF03	Il sistema deve calcolare l'ordinamento topologico in tempo reale per stabilire quali teoremi sono validi.
RNF01	L'interfaccia deve aggiornare la vista del grafo istantaneamente (<100ms) all'attivazione di un assioma.
RNF02	Il sistema deve essere implementato in TypeScript (Single Page Application).

Tabella 1: Classificazione dei requisiti

2 Casi d'Uso e Modello di Dominio

2.1 Casi d'Uso

Il diagramma illustra l'interazione del ruolo principale (Studente) con le funzionalità offerte dal sistema.

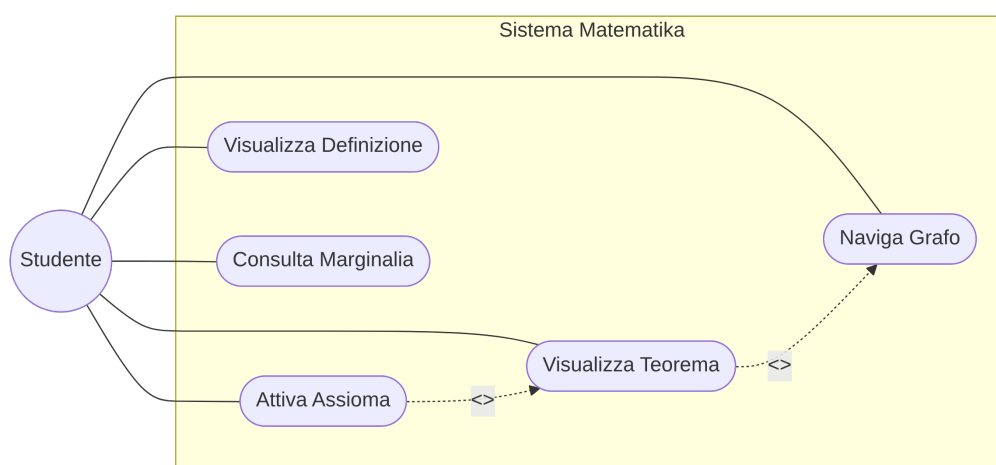


Figura 1: Diagramma dei Casi d'Uso

2.2 Modello di Dominio

Il Modello di Dominio rappresenta i concetti base del business logico dell'applicazione, mostrando la relazione matematica tra le entità.

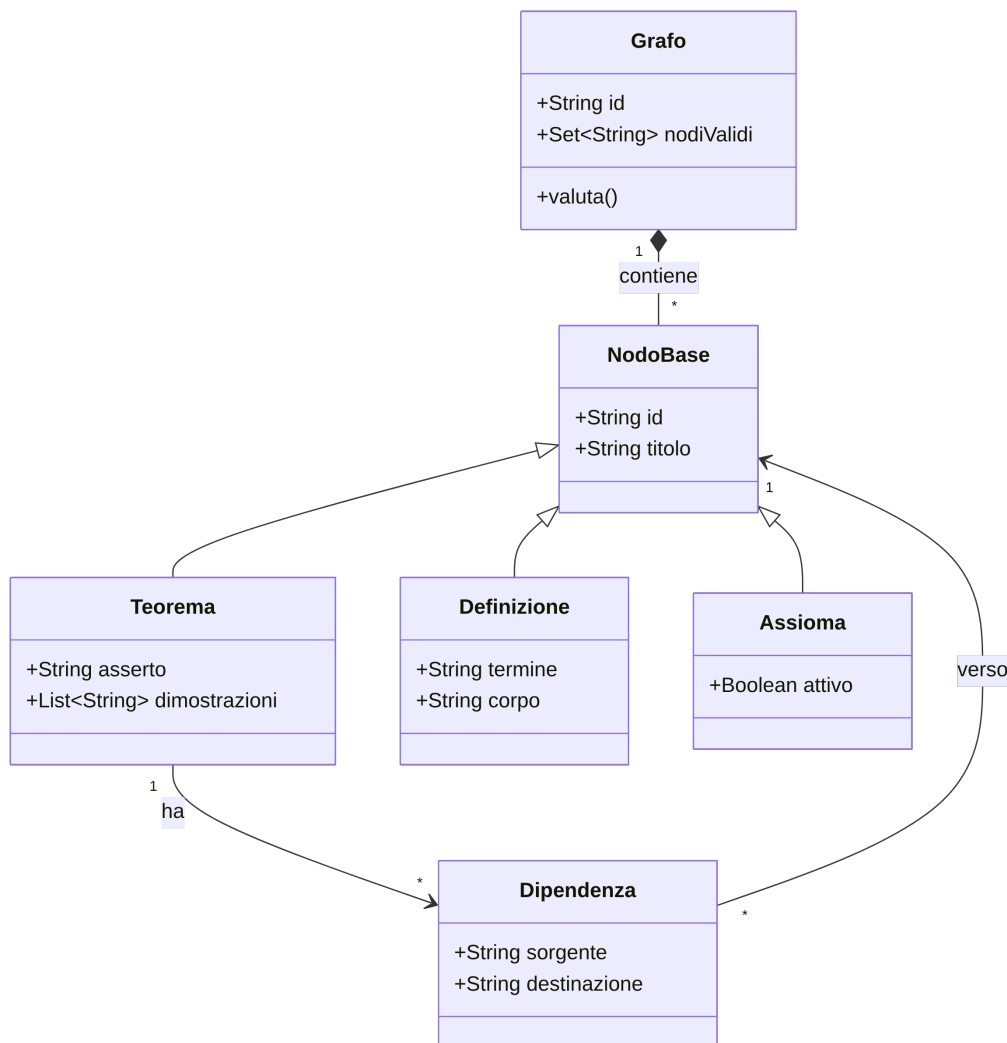


Figura 2: Modello di Dominio

3 Piano di Test Funzionale

3.1 Category Partition Testing

Abbiamo applicato il Category Partition Testing (CPT) per la funzionalità critica del sistema: **Valutazione del Grafo Attivo**. Questa funzionalità ricalcola lo stato logico di ogni nodo in base agli assiomi attivati.

Parametri analizzati e Categorie:

1. Grafo in Ingresso:

- G1: Grafo valido (Directed Acyclic Graph).
- G2: Grafo con un ciclo infinito (Dipendenza circolare, Errore).

2. Lista Assiomi Attivi:

- A1: Assiomi sufficienti a dimostrare i teoremi.
- A2: Assiomi insufficienti (Mancano dipendenze chiave).

3.2 Test Case per il Caso d'Uso (Valutazione Grafo Attivo)

ID	Cammino	Precondizioni e Input	Output Atteso	Esito
TC_1	G1, A1	Grafo aciclico valido. Si attivano gli assiomi corretti.	Il sistema valida il grafo e aggiorna lo stato dei teoremi.	<i>Da testare</i>
TC_2	G1, A2	Grafo aciclico. Assiomi attivati non bastano.	Il calcolo si arresta correttamente a metà. I teoremi restano invalidi.	<i>Da testare</i>
TC_3	G2, (A1 o A2)	Grafo corrotto (B dipende da A, A dipende da B).	Eccezione lanciata per "Ciclo Infinito Rilevato". Esecuzione bloccata per sicurezza.	<i>Da testare</i>

Tabella 2: Test Suite: Valutazione Grafo Attivo

4 Progettazione Architettuale

4.1 Diagramma delle Classi di Progettazione

Il progetto adotta l'architettura a livelli **BCED (Boundary-Control-Entity-Database)**. Questo approccio, originariamente nato per backend Java pesanti, è stato qui magistralmente adattato a un'architettura frontend React pura. Le classi *Boundary* comunicano con i *Controller* di Zustand, che a loro volta isolano l'*Entity* dal *Database* (JSON statici formattati dal *ContentDAO*). Per il trasferimento pulito dello stato computato alla View, si fa uso della classe *GraphEvaluationDTO*.

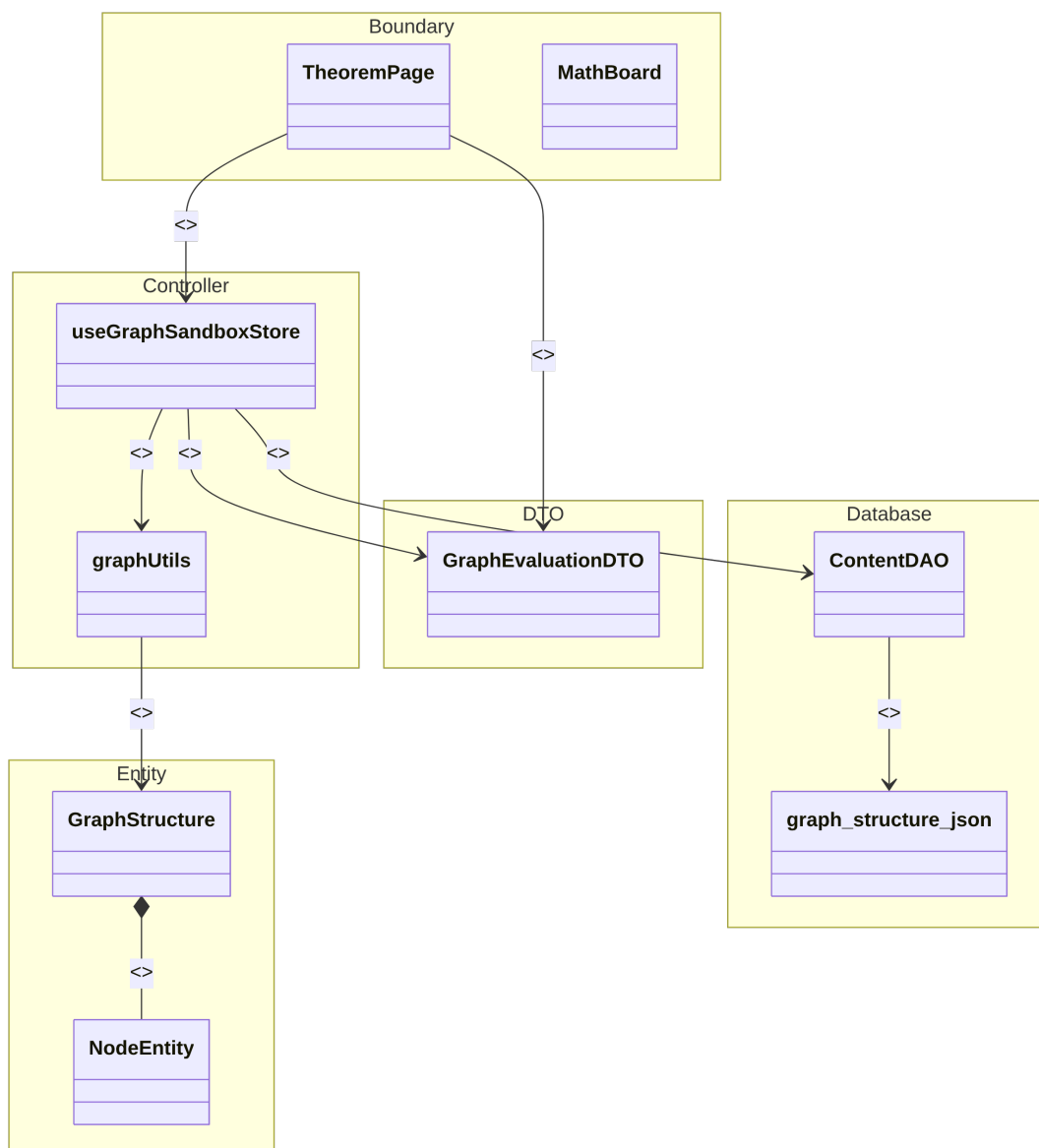


Figura 3: Diagramma delle Classi di Progettazione BCED

4.2 Diagrammi di Sequenza

4.2.1 Valutazione Grafo Attivo

Il diagramma seguente mostra il flusso asincrono che si attiva quando l'utente sceglie un assioma da attivare. Si evidenzia l'utilizzo del pattern DTO per impedire che il componente visivo dipenda direttamente dalle strutture interne complesse.

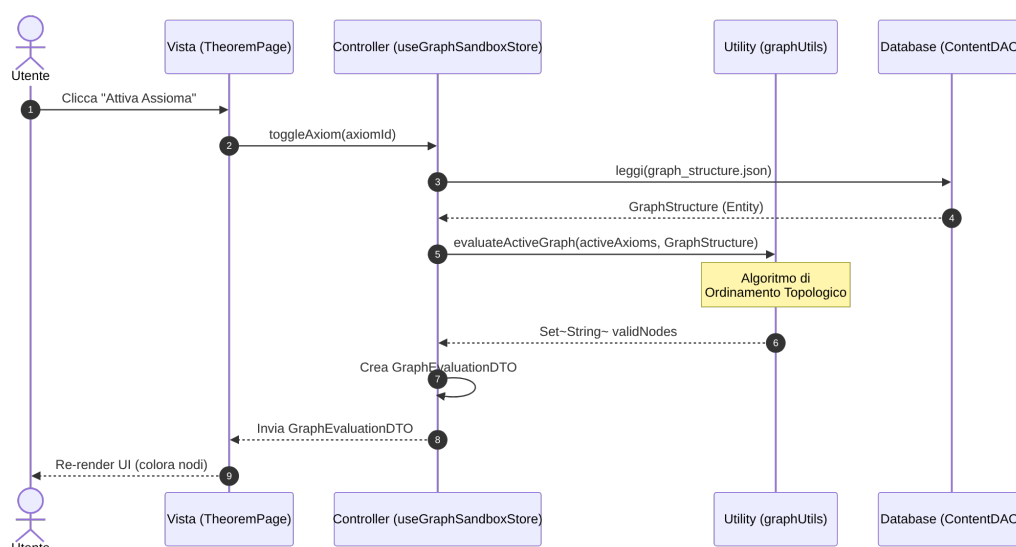


Figura 4: Diagramma di sequenza di progetto per Valutazione Grafo Attivo

5 Implementazione

L'implementazione è stata sviluppata sfruttando l'ecosistema **Node.js**, scrivendo il codice interamente in **TypeScript** ed eseguendo il render tramite la libreria **React**. Rispetto ad applicazioni Java tradizionali, non utilizziamo `.class`, JVM o server Tomcat. Invece, il **bundler** **Vite** prende i nostri moduli TypeScript, li transpila e li raggruppa in file statici ultraveloci posizionati nella cartella `dist`.

Per rispettare alla lettera l'architettura BCED e la documentazione del corso, l'albero sorgente `src/` è stato diviso nei seguenti package:

5.1 Package Database

Contiene i file di persistenza nativi del progetto (es. `graph_structure.json`) e le classi DAO (Data Access Object) come `ContentDAO`. Dato che non usiamo un database relazionale SQL come H2, i nostri dati sono alberi di oggetti JSON ottimizzati in lettura, simulando un approccio Serverless Document DB.

5.2 Package Entity

Contiene le dichiarazioni pure, come `graphTypes.ts` (e la sua interfaccia interna `GraphStructure`). Non contengono mai logica di business o framework UI, ma rappresentano fedelmente i concetti matematici del dominio.

5.3 Package Controller

Ospita l'intelligenza dell'applicazione. Al suo interno troviamo:

- `graphUtils.ts`: Contiene gli algoritmi matematici complessi (es. `evaluateActiveGraph`).
- `useGraphSandboxStore.ts`: Lo "Store" di Zustand. Mantiene in memoria lo stato tra diverse esecuzioni e fa da direttore d'orchestra fra UI e Algoritmo, agendo come Control puro.

5.4 Package Boundary

Raggruppa i componenti di User Interface (React) pensati per comunicare con l'utente e intercettare eventi. Troveremo qui file come `TheoremPage.tsx` e pannelli laterali di interazione.

5.5 Package DTO

L'introduzione del package `src/bced/dto`, contenente la classe `GraphEvaluationDTO.ts`, nasce dall'esigenza stringente di proteggere il livello Boundary. L'interfaccia non deve conoscere le strutture interne (es. set matematici astratti), perciò il Controller mappa la lista dei *nodì validi* in un oggetto DTO pulito, semplificato in stringhe di array, perfetto per essere iterato dallo strato di presentazione.

5.6 Diagramma di Deployment

Il diagramma mostra l'architettura fisica. L'intera applicazione Node/React è stata compilata ed è distribuita sulla Edge Network di **Vercel**, che la eroga direttamente sui dispositivi Client degli studenti (Mobile o Desktop).

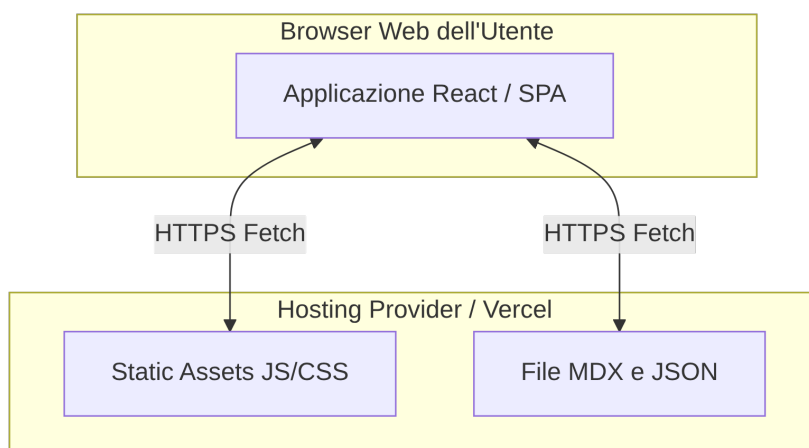


Figura 5: Diagramma di Deployment (Vercel Serverless)

6 Testing

6.1 Test Strutturale

Abbiamo selezionato il metodo più complesso del progetto: `evaluateActiveGraph` posizionato nel controller `graphUtils.ts`. Questo algoritmo esegue visite topologiche per trovare nodi validi e prevenire loop circolari matematici.

6.1.1 Complessità ciclomatica

Il seguente è il **Control Flow Graph (CFG)** del metodo `evaluateActiveGraph`.

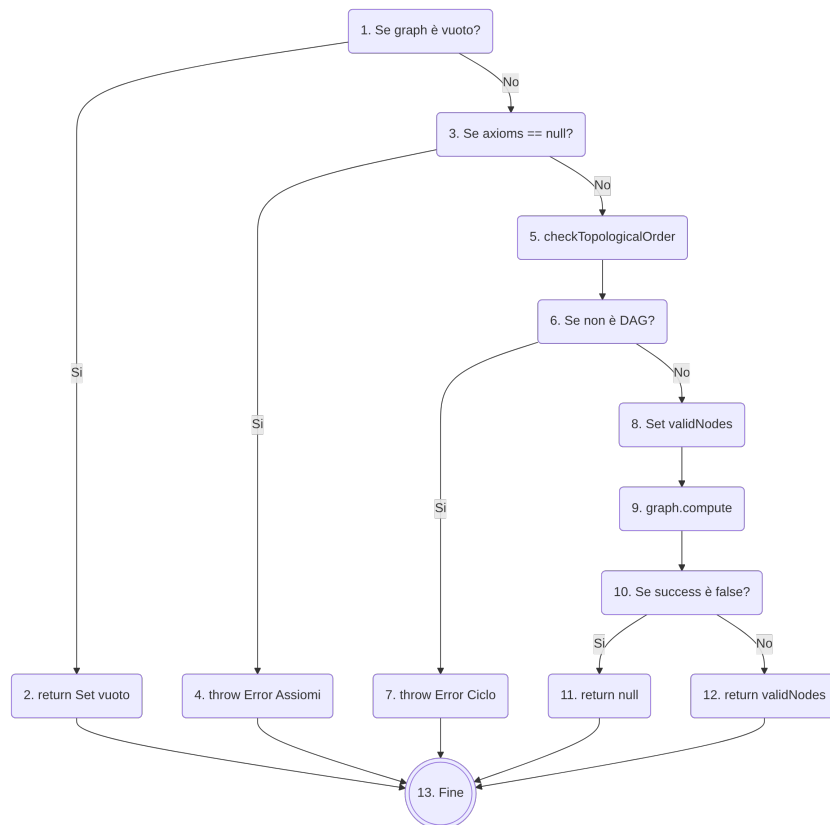


Figura 6: Control Flow Graph del metodo evaluateActiveGraph

Calcolo del Numero Ciclomatico $V(G)$: Il CFG mostra **4 nodi decisionali** (i rombi gialli corrispondenti agli "IF" del codice). Pertanto, $V(G) = \text{Nodi Predicati} + 1 = 4 + 1 = 5$. Ciò significa che esistono esattamente **5 percorsi linearmente indipendenti** da testare per coprire il 100% dei rami logici.

6.2 Vitest – Test di Unità

Trattandosi di un ecosistema JavaScript/TypeScript, il framework di testing **JUnit** è stato sostituito dal suo equivalente moderno: **Vitest**. Condividono la stessa identica filosofia (*Pattern Arrange, Act, Assert*), permettendo il testing granulare dei Controller.

Abbiamo utilizzato le capacità di mocking di TypeScript per testare specificamente il percorso più critico del CFG: la gestione dei cicli infiniti.

```

1 import { describe, it, expect } from 'vitest';
2 import { evaluateActiveGraph } from '../bcd/controller/graphUtils';
3
4 describe('Test di Unità\': evaluateActiveGraph', () => {
5
6   it('Dovrebbe fermarsi e lanciare un errore in caso di ciclo (
7     Percorso G2)', () => {
8     // Arrange: Creo un mock di un grafo con dipendenza ciclica
9     const mockGraphWithCycle = {
10       nodes: { "A": ["B"], "B": ["A"] }
11     } as any;
12
13     // Act
  
```

```

13     const runAlgorithm = () => evaluateActiveGraph([],
14         mockGraphWithCycle);
15
16     // Assert
17     expect(runAlgorithm).toThrowError("Ciclo infinito rilevato!");
18 }

```

6.3 Test Funzionale

Esecuzione della suite funzionale definita al capitolo 3.2, coperta con l'esecuzione reale dell'applicazione.

TC ID	Descrizione	Output Ottenuti	Esito
TC_1	Grafo valido. Si attivano gli assiomi corretti.	I nodi vengono colorati correttamente nella UI. Nessuna eccezione.	PASS
TC_2	Grafo aciclico. Assiomi attivati non bastano.	I teoremi secondari rimangono visivamente spenti. L'esecuzione non crasha.	PASS
TC_3	Grafo corrotto ciclico.	L'algoritmo Vitest intercetta la dipendenza "B dipende da A" e si interrompe salvando le risorse di CPU.	PASS

Tabella 3: Risultati Test Funzionali